

【前端面经】美图Meitu

一面

1. 做的主要是什么项目，桌面端的吗
2. 用的主要是什么技术栈
3. vue 有了解吗
4. 移动端开发一般怎么判断是什么平台，安卓、IOS、微信？
5. 移动端页面，页面的尺寸、宽高怎么去处理？
6. 怎么根据设计稿的尺寸计算出 rem
7. react
 - a. 函数式组件，在一个应用周期里，什么时机会被调用到呢，函数会被调用多少次
 - b. 什么情况下会触发组件更新？
 - c. 子组件没有任何的 props，父组件在渲染的时候，子组件会跟着渲染吗
 - d. React.memo 会做什么处理
 - e. 组件 return JSX，这个需要在编译的时候转化才能运行，在编译阶段会被转义成什么 JS 代码？
 - f. react 函数式组件，hooks 有一定的写法规范，是出于什么样的考虑，会有这样的限制？
 - g. react fiber 是在什么情况下诞生的，是为了解决什么问题
 - h. react 优化需要手动优化、有没有一些方案可以自动处理这个问题
8. 服务端渲染
 - a. 项目中有没有服务端渲染的项目，有没有了解过如果用 react 需要配合什么框架去做
 - b. 对服务端渲染原理了解吗，比如是怎么实现服务端渲染的？
 - c. 服务端要做什么处理，达成生成 DOM 结构的目的？
9. vue
 - a. 怎么实现响应式，依赖收集
10. react vue 性能方面的差异
11. vue 有 computed，react 是怎么处理的
12. 对除 JS 之外的其他语言有了解吗
13. 编程里进程和线程的概念
14. 一个进程可能会有几个堆几个栈

15. 堆内存和栈内存为什么会分成这两种，各自有什么不同
16. JS 是一个单线程的语言，它这个特点有什么好处吗，为什么要设计成单线程的？
17. nodejs 相较于其他的服务端，有什么优点吗？
18. js 运行需要什么
19. V8 是用什么语言写的，为什么要用
20. 高级语言需要什么样的步骤才能让计算机运行
21. 编译 C++ 代码的程序是用什么语言写的
22. 用 nodejs 写过什么吗
23. 事件循环能解释一下吗
 - a. 如果有一个 `setTimeout(()=>{}, 1000)`，这个回调是什么时候进入到队列里排队呢，是执行到 `setTimeout` 的时候，还是等 1000ms 之后呢
24. 事件循环，这些是谁在处理
25. 浏览器环境下，JS 执行在哪个进程哪个线程
26. 解释一下闭包
 - a. 有没有什么必要条件需要满足才能产生闭包
 - b. 如果有一个外部函数内部返回一个内部函数，内部函数引用了外部函数的变量，产生闭包，如果这个外部函数被调用 10 次，会产生多少个闭包呢？
 - c. 对其中一个闭包里的变量进行修改之后，会影响其他闭包里的变量吗
 - d. 有没有别的写法能生成闭包
27. 原型链继承方式
28. 递归的定义，用到的场景有哪些
29. 解释一下链表？
30. 深拷贝
31. 遍历对象的所有属性
 - a. `for...in...`
 - b. `Object.keys()`
32. CSS 定位有哪几种

二面

一直聊项目

三面 现场 领导面

1. 介绍项目经历
 - a. 遇到什么问题
 - b. 怎么解决的

HR 面

不完整记录

1. 离职的原因是什么
2. 你熟悉 VUE 吗？
3. 学习一些新技术，或者是业务中碰到的一些难题，你是针对性去解决会比较多一些，还是说你有系统的学习的一些规划呢？
4. 工作过程中哪些成长比较大，遇到过哪些难题，怎么解决的？
5. 你们的加班强度怎么样？

参考答案：

1. 前端性能优化方案（详细版）
2. 减少HTTP请求：
 - 合并CSS和JavaScript文件。
 - 使用CSS精灵图合并图片。
 - 使用Data URL内嵌小图片。
3. 使用CDN：
 - 将静态资源托管在CDN上，提高全球访问速度。
4. 文件压缩和最小化：
 - 使用工具（如Webpack）压缩和最小化JavaScript、CSS和HTML文件。
 - 移除不必要的空格、注释和调试代码。
5. 缓存策略：
 - 配置合理的缓存头，如 `Cache-Control` 和 `ETag`。
 - 使用Service Worker进行更高级的缓存管理。
6. 懒加载和按需加载：
 - 图片懒加载：使用 `<img loading="lazy">` 或Intersection Observer API。

- 按需加载模块：使用Webpack的代码分割和动态 `import`。

7. 优化图片：

- 使用合适的图片格式，如WebP。
- 压缩图片，使用工具如ImageOptim、TinyPNG。

8. 减少重绘和重排：

- 尽量减少DOM操作，使用Document Fragment和批量更新。
- 使用CSS3硬件加速（如transform和opacity）。

9. 预加载和预取：

- 使用 `<link rel="preload">` 预加载关键资源。
- 使用 `<link rel="prefetch">` 预取未来可能需要的资源。

10. 使用合适的前端框架和库：

- 使用轻量级框架和库，如Preact、Vue.js。
- 避免过度使用大型库，按需引入模块。

11. 其他优化：

- 使用HTTP/2提高并发请求效率。
- 减少DOM节点数量，优化HTML结构。
- 避免使用内联样式和脚本，尽量使用外部文件。

12. Vue有了解吗

是的，对Vue有了解，包括其核心概念、组件化开发、双向数据绑定、指令系统、路由管理（Vue Router）、状态管理（Vuex）等。

13. 移动端开发判断平台（安卓、iOS、微信）

```
1  function detectPlatform() {
2      const ua = navigator.userAgent;
3      if (/MicroMessenger/i.test(ua)) {
4          return 'WeChat';
5      } else if (/Android/i.test(ua)) {
6          return 'Android';
7      } else if (/iPhone|iPad|iPod/i.test(ua)) {
8          return 'iOS';
9      } else {
10         return 'Unknown';
11     }
12 }
```

14. 移动端页面尺寸、宽高处理

- 使用viewport meta标签：

```
1 <meta name="viewport" content="width=device-width, initial-scale=1">
```

- 使用百分比、vw、vh等相对单位：

```
1 .container {  
2   width: 100vw;  
3   height: 100vh;  
4 }
```

- 使用flexbox布局：

```
1 .container {  
2   display: flex;  
3   justify-content: center;  
4   align-items: center;  
5 }
```

15. 根据设计稿尺寸计算出rem

假设设计稿宽度为750px，根元素的font-size设置为设计稿宽度的1/10：

```
1 html {  
2   font-size: 75px; /* 750px / 10 */  
3 }  
4  
5 body {  
6   width: 10rem; /* 750px宽度的元素 */  
7 }
```

16. React相关问题

17. 函数式组件调用时机和次数：

- 在每次渲染时都会被调用，可能多次调用取决于父组件的重新渲染。

18. 触发组件更新的情况：

- `state` 或 `props` 的变化。
- 父组件的重新渲染。
- 调用 `forceUpdate()`。

19. 子组件没有任何props时，父组件渲染时是否重新渲染：

- 仍然会重新渲染，除非使用 `React.memo` 进行优化。

20. `React.memo`处理：

- 通过浅比较来决定是否重新渲染组件。

21. JSX在编译阶段转化成什么JS代码：

```
1  // JSX
2  const element = <h1>Hello, world!</h1>;
3
4  // 编译后的JS
5  const element = React.createElement('h1', null, 'Hello, world!');
```

1. Hooks写法规范的考虑：

- 确保Hooks在每次渲染中以相同的顺序调用，避免依赖不稳定的变量。

2. React Fiber诞生背景和问题：

- 为了提高React在处理大量更新时的性能和可中断渲染的能力。

3. React自动优化方案：

- 使用 `React.PureComponent`、`React.memo`。
- 使用 `reselect` 库进行选择器优化。

4. 服务端渲染（SSR）

5. React服务端渲染的框架：

- 使用Next.js框架。

6. 服务端渲染原理：

- 在服务器上生成HTML并发送到客户端，客户端接管后进行后续交互。

7. 服务器处理生成DOM结构：

- 使用ReactDOMServer的 `renderToString` 或 `renderToNodeStream` 方法生成HTML。

8. Vue响应式和依赖收集

- Vue通过数据劫持（使用 `Object.defineProperty`）实现响应式。
- 依赖收集：在getter中收集依赖，在setter中通知依赖更新。

9. React和Vue性能差异

- 初次渲染：Vue的模板解析更快，React的虚拟DOM更灵活。
- 更新机制：Vue的依赖追踪更细粒度，React的diff算法更高效。

10. React的computed处理

- 使用 `useMemo` 或 `useCallback` 实现类似Vue的 `computed` 属性。

11. 对JS以外的语言了解

对Python、Java、C++等语言有一定了解。

12. 进程和线程的概念

- 进程：操作系统分配资源的基本单位。
- 线程：CPU调度的基本单位，同一进程中的多个线程共享资源。

13. 一个进程可能有几个堆几个栈

- 一个进程有一个堆，多个栈（每个线程一个栈）。

14. 堆内存和栈内存的区别

- 堆内存：动态分配，大小不定，管理复杂，速度较慢。
- 栈内存：静态分配，大小固定，管理简单，速度较快。

15. JS单线程的好处

- 避免多线程带来的同步问题，简化编程模型。

16. Node.js的优点

- 非阻塞I/O，处理高并发性能优越。
- 统一的JavaScript语言栈，方便前后端共享代码。

17. JS运行需要

- JavaScript引擎（如V8）和运行环境（如浏览器或Node.js）。

18. V8引擎的语言

- V8引擎用C++编写。

19. 高级语言运行步骤

- 解析、编译、链接、生成可执行文件。

20. 编译C++代码的程序语言

- 通常用C或C++编写（如GCC）。

21. Node.js项目经验

有Node.js开发经验，包括API开发、文件处理、服务搭建等。

22. 事件循环

23. setTimeout回调的入队时机：

- 1000ms后回调进入队列。

24. 事件循环的处理者

- 由JavaScript引擎和运行环境（如浏览器或Node.js）共同处理。

25. 浏览器环境下JS执行

- 在浏览器的渲染进程的主线程上执行。

26. 闭包

27. 闭包必要条件：

- 函数内返回函数，并引用外部函数变量。

28. 多个调用的闭包数量：

- 每次调用产生一个新的闭包。

29. 闭包变量独立性：

- 变量在不同闭包中是独立的。

30. 其他闭包生成方式：

```
1  function createCounter() {  
2      let count = 0;  
3      return function() {  
4          return count++;  
5      };  
6  }
```

31. 原型链继承方式

- 使用构造函数和原型链实现继承。

32. 递归定义和场景

- 函数调用自身，常用于遍历树结构、计算阶乘、斐